

Fine-tune Navigation with Route Guards



Lara Newsom

Software Technical Leader | Speaker | Angular GDE

[lara-newsom](#) | www.linkedin.com



Angular Route Guards

CanActivate

CanActivateChild

CanDeactivate

CanMatch



Route guards do not provide real application security; they enhance the user experience by preventing empty views.





Implementing Route Guards

Route guards are declared on a route

Each guard type has its own route property

Guard properties accept an array of guards

Chaining guards allows multiple checks to fine-tune navigation



Generate a Route Guard with the CLI

```
% ng generate guard
```

✓ What name would you like to use for the guard?

`is-feature-enabled`

✓ Which type of guard would you like to create?

`CanMatch`

```
CREATE src/app/is-feature-enabled.guard.spec.ts
```

```
CREATE src/app/is-feature-enabled.guard.ts
```



Route Guards

Route Guards are functions that return a **GuardResult**

GuardResults can be a boolean, a **UrlTree**, or a **RedirectCommand**

Results can be returned synchronously or asynchronously

The router handles getting async values



Returning Booleans

true



Navigation continues

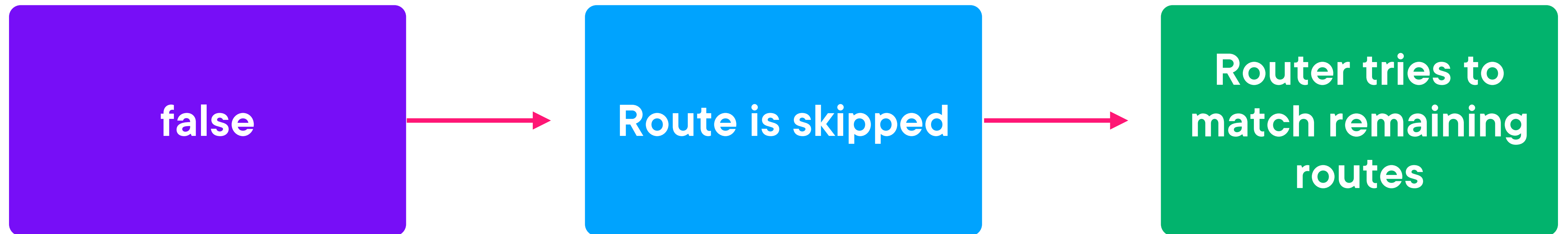
false



Navigation cancelled



Returning Booleans - CanMatch



Returning Redirect Instructions

UrlTree

Create a `UrlTree` using `createUrlTree` on the `RouterService`

Pass an array of route segments to provide the path

Pass optional `UrlCreationOptions` to add query parameters or relative routing

Initial navigation is cancelled and a new navigation event is started on the new `UrlTree`

vs.

RedirectCommand

Offers options in addition to simply returning a `UrlTree`

The `redirectTo` property accepts a `UrlTree`

Optional `navigationBehaviorOptions` can control how the router handles the browser history





Route guards provide snapshots of router details as arguments

Route guards run inside Angular's dependency injection context

Route details and injected services can be used inside route guard functions



Bethany's Pie Shop Pizza Pie Feature



A new feature that offers custom pizza pies

Only available to logged-in users

Behind a feature flag until it is released

Implement route guards to provide the expected user experience



CanActivate Route Guards



CanActivate Route Guard

Decides if a user is able to access a route

If it returns true, navigation continues

If it returns false, navigation is cancelled

Returning redirect instructions gives users a better experience for disallowed navigation



CanActivateFn

```
myGuard: CanActivateFn = (  
  // Arguments provide access to route details  
  route: ActivatedRouteSnapshot, ←  
  state: RouterStateSnapshot ←  
) => {  
  // Dependencies can be injected  
  const myService = inject(MyService); ←  
  
  return myService.isAuthenticated(); ←  
};
```



Demo: Creating Route Guards with the CLI



CanActivateChild Route Guards



Used to protect child routes on a parent route

Protects all children, including nested children

Its behavior and structure are nearly identical to CanActivate

Receives snapshot details for the child route instead of the parent route



Demo: CanMatch Route Guards





CanMatch Route Guards

Useful when more than one route has the same path

When it returns true, the route is considered for matching

When it returns false, the route is skipped and the router keeps trying to find a match

Executes before CanActivate guards

Exposes the route and an array of UrlSegments as arguments

Typically used to return a boolean



CanDeactivate Route Guards



**CanDeactivate route guards
are used to prevent accidentally
navigating away from an
incomplete workflow.**



CanDeactivateFn

```
myGuard: CanDeactivateFn = (  
  // Arguments provide access to the active  
  // component and route details  
  component: MyComponent, ←  
  currentRoute: ActivatedRouteSnapshot, ←  
  currentState: RouterStateSnapshot, ←  
  nextState: RouterStateSnapshot, ←  
) => {  
  // Component properties and methods are exposed  
  if(component.canSubmit()) { ←  
    component.showWarning(); ←  
    component.nextNav.set(nextState.url); ←  
    return false;  
  }  
  
  return true;  
};
```



Returning Booleans - CanDeactivate

